

Web Scrapping Notes for Basic Understanding

What is Web Scrapping? Extracting structured data from websites.

When to Use? Collecting data for research, automation, or analysis.

Legal Considerations: Always check the website's robots.txt file before scraping.

Example: Check `robots.txt` for Wikipedia

URL: <https://www.wikipedia.org/robots.txt>

Q.What is difference between Scrapy, BeautifulSoup and Selenium?

Feature	Scrapy	BeautifulSoup	Selenium	
Type	Framework	Library	Automation Tool	
Speed	Fast	Moderate	Slow	
JavaScript Support	No	No	Yes	
Best Use Case	Large sites	Simple parsing	Dynamic JS pages	
Learning Curve	Moderate	Easy	Moderate	

What is difference between crawler and scraper?

Feature	Web Crawler 🕷	Web Scraper ✂	
Purpose	Finds and indexes webpages	Extracts specific data	
Works on	Multiple sites/pages	Single/multiple pages	
Example Use	Googlebot indexing sites	Scraping product prices	
Tools Used	Scrapy, Selenium	BeautifulSoup, Requests	

- A **crawler** discovers and indexes pages.
- A **scraper** extracts specific information.

BeautifulSoup Library Syntaxes

Install Required Libraries:

- **requests** – Fetch webpage HTML
 - **BeautifulSoup** – Parse HTML
 - **pandas** – Store data in a structured format
- ⇒ `pip install requests beautifulsoup4 pandas`

Fetching a Webpage:

We use requests to download HTML content.

Example: Fetching Wikipedia's Homepage

```
import requests
url = "https://www.wikipedia.org/"
response = requests.get(url)
print(response.status_code)  # 200 means success
print(response.text[:500])  # Print first 500 characters of HTML
```

Parsing HTML with BeautifulSoup:

Once we have the HTML, we parse it to extract meaningful data.

Example: Extracting Page Title

```
from bs4 import BeautifulSoup
soup = BeautifulSoup(response.text, "html.parser")
print(soup.title.text)  # Outputs: "Wikipedia"
```

Extracting Different Types of Objects in BeautifulSoup:

- The main entry point that represents the entire HTML document.
- Created using `BeautifulSoup(html, "html.parser")`.

Example:

```
from bs4 import BeautifulSoup
html = "<html><head><title>My  
Page</title></head><body>Hello!</body></html>"
soup = BeautifulSoup(html, "html.parser")
```

```
print(type(soup)) # <class 'bs4.BeautifulSoup'>
print(soup.prettify()) # Nicely formatted HTML
```

2. Tag Object

- Represents an individual HTML/XML tag.
- Used to extract elements like <title>, <p>, <div>, etc.

Example:

```
print(soup.title) # <title>My Page</title>
print(soup.title.name) # title
print(soup.title.string) # My Page
```

3. NavigableString Object

- Represents **text inside a tag**.

Example:

```
title_tag = soup.title
print(title_tag.string) # My Page
print(type(title_tag.string)) # <class 'bs4.element.NavigableString'>
```

4. Comment Object

- Used to extract **HTML comments** from a page.

Example:

```
html = "<p><!-- This is a comment --></p>"
soup = BeautifulSoup(html, "html.parser")
comment = soup.p.string
print(comment) # This is a comment
print(type(comment)) # <class 'bs4.element.Comment'>
```

5. ResultSet Object

- A list-like collection returned by `find_all()` when searching for multiple tags.

Example:

```
html = "<p>First</p><p>Second</p>"
soup = BeautifulSoup(html, "html.parser")
paragraphs = soup.find_all("p")
print(paragraphs) # [<p>First</p>, <p>Second</p>]
print(type(paragraphs)) # <class 'bs4.element.ResultSet'>
```

6. Attribute Object

- Used to access **tag attributes** (like href, class, id).

Example:

```
html = '<a href="https://example.com" class="link">Click me</a>'
soup = BeautifulSoup(html, "html.parser")
link = soup.a
print(link['href']) # https://example.com
print(link['class']) # ['link']
```

What is difference between find() and find_all()?

find()

- Returns **only the first matching element**.
- If no match is found, it returns None.
- Used when you are looking for a **single** occurrence of an element.
- **Example:**

```
first_p = soup.find('p') # Finds the first <p> tag
print(first_p.text) # Output: Hello
```

find_all()

- Returns **a list of all matching elements**.
- If no match is found, it returns an empty list [].
- Used when you need **multiple** occurrences of an element.
- **Example:**

```
all_p = soup.find_all('p') # Finds all <p> tags
for p in all_p:
    print(p.text)
```

Example: Extracting All Links (<a> Tags)

```
import requests
from bs4 import BeautifulSoup
# URL to scrape
url = "http://quotes.toscrape.com/"
# Fetch the webpage
response = requests.get(url)
# Parse HTML
soup = BeautifulSoup(response.text, "html.parser")
# Find all <a> tags and extract href attributes
links = [a["href"] for a in soup.find_all("a", href=True)]
# Print all links
for link in links:
    print(link)
```

Extract All Image URLs (Tags):

```
from bs4 import BeautifulSoup
import requests
# Fetch and parse the webpage
url = "http://quotes.toscrape.com/"
response = requests.get(url)
soup = BeautifulSoup(response.text, "html.parser")
# Find all <img> tags and extract the "src" attribute
images = [img["src"] for img in soup.find_all("img")]
# Print all image URLs
for img_url in images:
    print(img_url)
```

Extract Images with a Specific Class or Attribute:

If images are inside a <div class="gallery">, use:

```
gallery_images = [img["src"] for img in soup.find("div",
class_="gallery").find_all("img")]
```

Or extract images with a specific class:

```
images = [img["src"] for img in soup.find_all("img", class_="product-image")]
```

Finding all span elements inside div:

```
data1 = soup.find("div", class_="w3-col l2 m12")
print(data1)

# Check if data1 is found
if data1:
    # Find all <span> elements inside the div
    spans = data1.find_all("span")

    # Iterate over found span elements and print their text
    for span in spans:
        print(span.text)
else:
    print("No matching <div> found.")
```

Scraping and storing results in csv file:

```
import requests
from bs4 import BeautifulSoup
import pandas as pd
```

```

# Define URL
url = "https://books.toscrape.com/"

# Send GET request
response = requests.get(url)
soup = BeautifulSoup(response.text, "html.parser")

# Find all product containers
books = soup.find_all("article", class_="product_pod")

# Create separate lists for titles and prices
titles = []
prices = []

# Loop through each book and store data
for book in books:
    title = book.h3.a["title"]
    price = book.find("p", class_="price_color").text
    titles.append(title) # Add to titles list
    prices.append(price) # Add to prices list

# Convert to DataFrame
df = pd.DataFrame({"Title": titles, "Price": prices})

# Save to CSV
df.to_csv("books.csv", index=False)

print("✅ Data saved to books.csv")
df = pd.read_csv("books.csv")

```

Scraping Multiple Pages:

Scraping data from multiple pages involves iterating through each page's URL and extracting the desired information.

Basic Steps:

1. **Inspect the pagination pattern** on the website.
2. **Use a loop** to go through each page.
3. **Parse the HTML** using BeautifulSoup.
4. **Extract the data** from each page.
5. **Store or process the data.**

Example:

```
import requests
from bs4 import BeautifulSoup
base_url = 'https://example.com/products?page='
all_data = []
# Define how many pages to scrape
for page in range(1, 6): # Scrape pages 1 to 5
    url = base_url + str(page)
    print(f"Scraping: {url}")

    response = requests.get(url)
    soup = BeautifulSoup(response.text, 'html.parser')
    # Example: Extract product names from <h2 class="product-title">
    products = soup.find_all('h2', class_='product-title')
    for product in products:
        title = product.text.strip()
        all_data.append(title)

print("Scraped Products:")
for item in all_data:
    print(item)
```

When No more items are found:

```
import requests
from bs4 import BeautifulSoup
base_url = 'https://example.com/products?page='
all_data = []
page = 1
while True:
    url = base_url + str(page)
    print(f"Scraping: {url}")
    response = requests.get(url)
    soup = BeautifulSoup(response.text, 'html.parser')
    # Find all product entries
    products = soup.find_all('h2', class_='product-title')
    if not products:
        print("No more products found. Stopping.")
        break # Exit the loop if no products found
    for product in products:
        title = product.text.strip()
        all_data.append(title)
    page += 1 # Move to next page
print(f"\nTotal products scraped: {len(all_data)}")
```

Selenium Library Syntaxes

What is Selenium?

Selenium is a tool that automates browsers. Can use it to write scripts that:

- Open websites
- Click buttons
- Fill out forms
- Scrape data
- Test web applications

Download WebDriver

Selenium needs a **WebDriver** to control a browser.

- For Chrome: ChromeDriver
- For Firefox: GeckoDriver

What is a WebDriver?

A **WebDriver** is like a **remote control for your web browser**.

It allows **Selenium (your Python code)** to open, control, and test a web browser **just like a human would**.

Think of it like this:

- 🧑 You = Programmer
- 💻 Selenium = Your assistant
- 🌐 Chrome browser = The screen you want to control
- ⚙️ ChromeDriver (WebDriver) = The *remote control* that lets Selenium talk to Chrome

Why Do We Need WebDriver?

Selenium can't directly open or control browsers like Chrome or Firefox.

So it needs help — and that help comes in the form of a **WebDriver**.

Example:

Searching Something on Google

```
from selenium import webdriver
from selenium import webdriver
from selenium.webdriver.common.by import By
import time
# Launch Chrome
driver = webdriver.Chrome()
# Open a website
driver.get("https://www.google.com")

# Find the search box using name attribute
search_box = driver.find_element(By.NAME, "q")

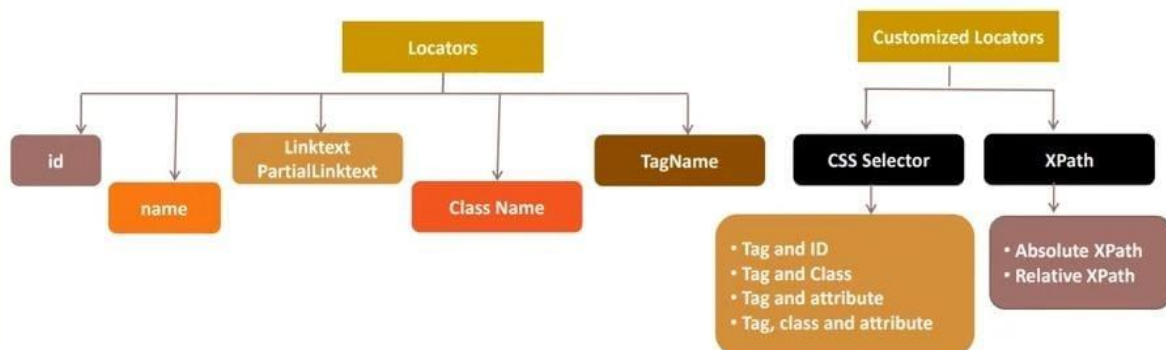
# Type in the search box
search_box.send_keys("Selenium Python")

# Submit the form
search_box.submit()
# Wait to see results
time.sleep(3)

# Close the browser
driver.quit()
```

Selenium Web Element Locators/Types of locator

- We can identify various elements on the web using **Locators**.
- Locators are addresses that identify a web element uniquely within the page.





Method	Description	Example in Python
By.ID	Selects by ID attribute	<code>driver.find_element(By.ID, "username")</code>
By.NAME	Selects by name attribute	<code>driver.find_element(By.NAME, "email")</code>
By.CLASS_NAME	Selects by class	<code>driver.find_element(By.CLASS_NAME, "login-box")</code>
By.TAG_NAME	Selects by tag name (like h1, a)	<code>driver.find_element(By.TAG_NAME, "h1")</code>
By.LINK_TEXT	Full text of a link	<code>driver.find_element(By.LINK_TEXT, "Click here")</code>
By.PARTIAL_LINK_TEXT	Partial text of a link	<code>driver.find_element(By.PARTIAL_LINK_TEXT, "Click")</code>
By.CSS_SELECTOR	Uses CSS selector (like in CSS/HTML)	<code>driver.find_element(By.CSS_SELECTOR, "#login input")</code>
By.XPATH	Uses XPath (advanced and powerful)	<code>driver.find_element(By.XPATH, "//input[@type='text']")</code>

Basic Commands

Action	Command Example
Open URL	<code>driver.get("https://example.com")</code>
Find element by name	<code>driver.find_element(By.NAME, "q")</code>
Find element by ID	<code>driver.find_element(By.ID, "email")</code>
Click a button	<code>button.click()</code>
Send keys (type text)	<code>input.send_keys("Hello")</code>
Get text from element	<code>element.text</code>
Close the browser	<code>driver.quit()</code>

What is XPath?

XPath stands for **XML Path Language**. It's a way to *navigate* through elements and attributes in an HTML or XML document.

XPath is defined as XML path.

It is a syntax or language for finding any element on the web page using XML path expression.

XPath is used to find the location of any element on a webpage using **HTML DOM** structure.

XPath can be used to navigate through elements and attributes in DOM.

XPath is an address of the element.

In Selenium, we use XPath to **find elements on a web page**, especially when:

- there's **no ID or class**
- you want to find a **specific element inside another**
- you need more **precise control**

DOM – Document Object Model

- DOM is an API Interface provided by browser.
- When a web page is loaded, the browser creates a Document Object Model of the page.

HTML

```
<!DOCTYPE html>
<html>

<head> </head>

<body>
  <button id="myBtn">Click Me</button>
  <input type="text" />
  <p id="demo1"> This is static text message </p>
  <p id="demo2"> Hello!</p>
</body>
</html>
```

DOM View

```
DOCTYPE: html
HTML
├── HEAD
│   └── #text:
├── #text:
├── BODY
│   ├── #text:
│   ├── BUTTON id="myBtn"
│   │   └── #text: Click Me
│   ├── #text:
│   ├── INPUT type="text"
│   ├── #text:
│   ├── P id="demo1"
│   │   └── #text: This is static text message
│   ├── #text:
│   ├── P id="demo2"
│   │   └── #text: Hello!
│   └── #text:
```

Rendered View

Click Me

This is static text message

Hello!

XPath works here

Basic XPath Syntax

```
//tagname[@attribute='value']
```

Examples:

```
//input[@id='username']    → Finds input tag with id="username"
//button[@type='submit']  → Finds submit button
//a[text()='Login']       → Finds a link with exact text "Login"
```

- `<input id="user" name="username" type="text" />`
- `<input class="password" type="password" />`
- `<button type="submit">Login</button>`

Types of xpath

1) Absolute/Full xpath

Ex: /html/body/nav/div/div[2]/ul[3]/li[1]/a
/html/body/div[1]/div/div[3]/div[1]/img

2) Relative/Partial xpath

Ex: `//*[@id="header-navbar"]/ul[3]/li[1]/a`
`//*[@id="divLogo"]/img`

Diff between Absolute & Relative Xpaths

- 1) Absolute xpath starts from root html node
Relative xpath directly jump to element on DOM.
- 2) Absolute xpath start with /
Relative xpath start with //
- 3) in Absolute xpath we use only tags/nodes
In Relative xpath we use attributes.

How to capture xpath automatically

- 1) Firebug, firepath --> firefox --> deprecated/ not available
- 2) Right click on element-->Inspect--> Highlight html code-->Right click-->Copy Xpath
- 3) SelectorsHub

2 reasons to prefer relative xpath

- 1) if developer introduced new element then absolute xpath will be broken.
- 2) if developer changed the location then absolute xpath will be broken.

absolute xpath is unstable....

Which XPath is preferred? Why?

Ans: Relative

xpath options

and

or

contains()

I

starts-with()

text()

What is a CSS Selector?

A **CSS Selector** is a way to target specific HTML elements using their:

- Tag name (div, p, span, etc.)
- Class (.classname)
- ID (#idname)
- Attribute ([name='value'])
- And you can even combine them.

```
1) tag id      tagname#valueofId    input#email
2) tag class   tagname.valueofClass input.inputtext _55r1_6luy
3) tag attribute tagname[attribute=value]    input[data-testid=royal_email]
4) tag class attribute tagname.valueofClass[attribute=value]
```

Basic Syntax¹

Selector Type	CSS Selector Example	What It Selects
Tag	Div	All <div> elements
Class	.quote	All elements with class="quote"
ID	#main	Element with id="main"
Tag + Class	div.quote	<div> elements with class quote
Attribute	input[name='username']	An input field with name="username"
Starts with attribute	a[href^='https']	All links starting with https
Contains text (not CSS)	use XPath instead	CSS can't directly check text content

What You Want to Select CSS Selector

All links	a
All buttons	Button
Input with placeholder="Search"	input[placeholder='Search']
All <div> with class "box"	div.box
A div with both box and highlight	div.box.highlight
ID called login-form	#login-form
Inside a parent: author inside quote	.quote .author

Example:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
driver = webdriver.Chrome()
driver.get("http://quotes.toscrape.com")
# Using CSS selector to find all author names
authors = driver.find_elements(By.CSS_SELECTOR, ".author")
for author in authors:
    print(author.text)
driver.quit()
• .author → Finds all elements with class "author"
```

Clicking a button

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time
# Start Chrome
driver = webdriver.Chrome()
# Open the page
driver.get("https://www.w3schools.com/html/html_forms.asp")
time.sleep(3) # wait for the page to load
try:
    # Click the "Try it Yourself" button
    try_it_button = driver.find_element(By.LINK_TEXT, "Try it Yourself »")
    try_it_button.click()
    time.sleep(5) # wait to see the new tab or page
except Exception as e:
    print("Error:", e)
finally:
    driver.quit() # close at the end, not before
```

Filling out a form

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

driver = webdriver.Chrome()
driver.get("https://www.w3schools.com/html/html_forms.asp")

first_name = driver.find_element(By.ID, "fname")
first_name.clear()
first_name.send_keys("Hira")

last_name = driver.find_element(By.ID, "lname")
last_name.clear()
last_name.send_keys("Nawaz")

time.sleep(5)
driver.quit()
```

Handle Alerts/Popups

```
from selenium import webdriver
import time
driver = webdriver.Chrome()
driver.get("https://the-internet.herokuapp.com/javascript_alerts")2
# Click the alert button
driver.find_element(By.XPATH, "//button[text()='Click for JS Alert']").click()
# Switch to alert
alert = driver.switch_to.alert
print("Alert text:", alert.text)
alert.accept() # or alert.dismiss()
time.sleep(2)
driver.quit()
```


Try Scraping a Website

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

driver = webdriver.Chrome()
driver.get("http://quotes.toscrape.com")

quotes = driver.find_elements(By.CLASS_NAME, "text")
authors = driver.find_elements(By.CLASS_NAME, "author")

for i in range(len(quotes)):
    print(f"{quotes[i].text} - {authors[i].text}")

time.sleep(2)
driver.quit()
```

Build a Test Case for Login

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

driver = webdriver.Chrome()
driver.get("https://the-internet.herokuapp.com/login")

driver.find_element(By.ID, "username").send_keys("tomsmith")
driver.find_element(By.ID, "password").send_keys("SuperSecretPassword!")
driver.find_element(By.XPATH, "//button[@type='submit']").click()

# Wait and verify login success
time.sleep(3)
message = driver.find_element(By.ID, "flash").text
print("Login message:", message)
driver.quit()
```

Basic Commands:

Application commands

get() - opening the application URL

title - to capture the title of the current webpage

current_url - to capture the current url of the web page

page_source - to capture source code of the page

Conditional commands

is_displayed()

is_enabled()

is_selected()

Browser commands

close() - close single browser window (where driver focused)

quit() - close multiple browser windows (this will kill the process)

Navigational commands

back()

forward()

refresh()

```
find_element() Vs find_elements()
```

```
text Vs get_attribute('value')
```

```
text ----> returns inner text of the element
```

```
get_attribute() ----> returns values of any attribute of web element
```

```
<input id='123' name="xyz"> Email: </input> ----> Email is inner text
```

What is "Wait" in Selenium?

When Selenium runs, sometimes **the page is still loading or elements are not ready yet**. If Selenium moves too fast, it might **fail** saying:

"Element not found!" ❌

To **avoid this problem**, we tell Selenium:

"Hey, **wait** for a few seconds before you continue."

This is called **Wait**.

There are **two types of Waits**:

Type	Meaning (Simple)
Implicit Wait	Wait for some time for everything you search on the page.
Explicit Wait	Wait for a specific thing (specific button, image, text) until it appears.

1. Implicit Wait

Implicit wait is like telling Selenium:

"**Always** wait a little bit if you don't find something immediately."

1. Applies to **everything**
2. Only needs to set **once**

How to write Implicit Wait in Python:

```
from selenium import webdriver
driver = webdriver.Chrome()
driver.implicitly_wait(5) # ← Waits up to 5 seconds for everything
driver.get("http://example.com")
```

```
# Selenium will wait up to 5 seconds when finding any element
element = driver.find_element(By.ID, "some-id")
```

Even if the element appears after 2 seconds, Selenium **doesn't waste time**, it continues immediately!

2. Explicit Wait

Explicit wait is like saying:

"**Only wait** for this **specific element** to appear."

1. More powerful
2. You can wait differently for different elements

How to write Explicit Wait in Python:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
driver = webdriver.Chrome()
driver.get("http://example.com")
# Explicitly wait up to 10 seconds for a specific element
element = WebDriverWait(driver, 10).until(
    EC.presence_of_element_located((By.ID, "some-id"))
)
# Now it found the element safely!
print(element.text)
```

Here, Selenium keeps checking **every second** until it finds the element (or until 10 seconds are over).

Quick Comparison Table:

Feature	Implicit Wait	Explicit Wait
Applies to	All elements	Only specific elements
How to set	One time	Every time for each element
Example use	Slow loading pages	Waiting for popups, buttons etc
Control	Less control	Full control

Simple Example for Life:

- **Implicit Wait** is like saying:
“I will wait at every shop for 5 minutes if the shopkeeper is not there.”
- **Explicit Wait** is like saying:
“I will wait exactly at the coffee shop until the coffee is ready.”

Summary:

If you want to...

Wait a little for **everything**

Wait specially for **one thing**

Use

Implicit Wait

Explicit Wait

Checkbox and Dropdown:

1. How to check a Checkbox?

Suppose you have a checkbox like:

- `<input type="checkbox" id="accept" name="accept">`

```
from selenium import webdriver
from selenium.webdriver.common.by import By
driver = webdriver.Chrome()
driver.get("https://example.com") # Change to your URL
# Find the checkbox
checkbox = driver.find_element(By.ID, "accept")
# Click to select it
checkbox.click()
```

Find total number of checkboxes:

```
checkboxes = driver.find_elements(By.NAME, "hobby")
total = len(checkboxes)
```

How to check if the checkbox is already selected?

```
if not checkbox.is_selected():
    checkbox.click()
```

How to select multiple checkboxes?

```
<input type="checkbox" name="hobby" value="cricket">
<input type="checkbox" name="hobby" value="football">
<input type="checkbox" name="hobby" value="reading">
```

```
from selenium import webdriver
from selenium.webdriver.common.by import By
driver = webdriver.Chrome()
driver.get("https://example.com") # Change to your URL
# Find all checkboxes with name="hobby"
checkboxes = driver.find_elements(By.NAME, "hobby")
# Click all checkboxes one by one
for checkbox in checkboxes:
    if not checkbox.is_selected(): # Check if not already selected
        checkbox.click()
```

Code to select first 2 checkboxes:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
driver = webdriver.Chrome()
driver.get("https://example.com") # Your page URL
# Find all checkboxes with name="hobby"
checkboxes = driver.find_elements(By.NAME, "hobby")
# Select only first 2
for checkbox in checkboxes[:2]: # [:2] means first two items
    if not checkbox.is_selected():
        checkbox.click()
```

Code to select last 2 checkboxes:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
driver = webdriver.Chrome()
driver.get("https://example.com") # Replace with your real URL
# Find all checkboxes
checkboxes = driver.find_elements(By.NAME, "hobby")
# Select the last two checkboxes
for checkbox in checkboxes[-2:]: # [-2:] means last two items
    if not checkbox.is_selected():
        checkbox.click()
```

Use range() for last two:

```
# Select the last two checkboxes using range
for i in range(total-2, total):
    if not checkboxes[i].is_selected():
        checkboxes[i].click()
```

Select even checkboxes (0, 2, 4, ...):

```
checkboxes = driver.find_elements(By.NAME, "hobby")
total = len(checkboxes)

# Select even index checkboxes
for i in range(0, total, 2): # Start from 0, jump by 2
    if not checkboxes[i].is_selected():
        checkboxes[i].click()
```

Select odd checkboxes (1, 3, 5, ...):

```
checkboxes = driver.find_elements(By.NAME, "hobby")
total = len(checkboxes)

# Select odd index checkboxes
for i in range(1, total, 2): # Start from 1, jump by 2
    if not checkboxes[i].is_selected():
        checkboxes[i].click()
```

What if you want to select only specific ones?

- Click "cricket" and "reading", but not "football"

```
for checkbox in checkboxes:
    value = checkbox.get_attribute("value")
    if value in ["cricket", "reading"]:
        if not checkbox.is_selected():
            checkbox.click()
```


How to scrape (print) all options from a dropdown?

```
<select id="country">
  <option value="pakistan">Pakistan</option>
  <option value="india">India</option>
  <option value="usa">USA</option>
</select>
```

```
from selenium.webdriver.support.ui import Select
dropdown = Select(driver.find_element(By.ID, "country"))
# Get all options
options = dropdown.options
# Print all options
for option in options:
    print(option.text)
```

How to select an item from a Dropdown?

Suppose you have a dropdown like:

```
<select id="country">
  <option value="pakistan">Pakistan</option>
  <option value="india">India</option>
  <option value="usa">USA</option>
</select>
```

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import Select
driver = webdriver.Chrome()
driver.get("https://example.com") # Change to your URL
# Find the dropdown
dropdown = Select(driver.find_element(By.ID, "country"))
# Select by visible text
dropdown.select_by_visible_text("Pakistan")
# You can also select by value
# dropdown.select_by_value("usa")
# Or select by index (starts from 0)
# dropdown.select_by_index(2)
```

Dropdown last two options:

```
from selenium.webdriver.support.ui import Select
dropdown = Select(driver.find_element(By.ID, "country"))
# Get all options
options = dropdown.options
# Select the last two options
dropdown.select_by_index(len(options) - 2) # second last
dropdown.select_by_index(len(options) - 1) # last
```

Dropdown (select first 2 options):

```
from selenium.webdriver.support.ui import Select
dropdown = Select(driver.find_element(By.ID, "country"))
# Get all options
options = dropdown.options
# Select the first two options (by index)
dropdown.select_by_index(0) # First option
dropdown.select_by_index(1) # Second option
```

What is an Alert box (Popup)?

👉 An alert box is a small popup that appears on a web page.

👉 It can show a **message**, **warning**, or ask you to **confirm something**.

Example you might have seen:

- "Are you sure you want to delete?"
- "Form submitted successfully!"
- "Please confirm your action."

It **blocks** the page — you **cannot** interact with the page until you handle (accept/cancel) the alert.

How to Handle Alerts in Selenium?

Use Selenium's `switch_to.alert` to control the alert box.

Accept the alert (click OK):

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time
driver = webdriver.Chrome()
driver.get("https://example.com") # Replace with real site
# Wait for alert to appear
time.sleep(2) # Just for demo, later you can use better waits
# Switch to alert
alert = driver.switch_to.alert
# Accept the alert (click OK)
alert.accept()
```

Dismiss the alert (click Cancel):

```
alert = driver.switch_to.alert
alert.dismiss()
```

Send keys (text) to Alert (only if it's a prompt alert):

```
alert = driver.switch_to.alert
alert.send_keys("Hello!")
alert.accept()
```

How to Handle Radio Buttons in Selenium?

1. **Find the radio button** using CSS selector, ID, name, XPath, etc.
2. **Click** on it using `.click()`.

Example:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time
driver = webdriver.Chrome()
driver.get("https://example.com") # Replace with your website
time.sleep(2)
# Select the "Female" radio button
female_radio = driver.find_element(By.XPATH, "//input[@value='female']")
female_radio.click()
```

Advanced User Actions :

1. Mouse hover (move to element)
2. Drag and Drop
3. Double Click
4. Right Click

1. Mouse Hover (Move to Element):

Now a **submenu** might open if you hover!

(no click, just move your mouse)

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver import ActionChains
import time
driver = webdriver.Chrome()
driver.get('https://example.com') # Your website
menu = driver.find_element(By.ID, "menu_id") # Find the menu
actions = ActionChains(driver)
actions.move_to_element(menu).perform()
time.sleep(2)
```

2. Drag and Drop

👉 Drag an item from one place and drop to another place.

```
source = driver.find_element(By.ID, "draggable")
target = driver.find_element(By.ID, "droppable")
actions = ActionChains(driver)
actions.drag_and_drop(source, target).perform()
```

3. Double Click

👉 Double click on a button, box, etc.

```
button = driver.find_element(By.ID, "double_click_btn")
actions = ActionChains(driver)
actions.double_click(button).perform()
```

- Some sites change color, text or open alerts on double-click!

4. Right Click (Context Click)

👉 Right click on an element (open right-click menu).

```
box = driver.find_element(By.ID, "right_click_box")
actions = ActionChains(driver)
actions.context_click(box).perform()
```

Taking Screenshots:

Save full page screenshot:

```
driver.save_screenshot('fullpage.png')
```

Save screenshot when test fails:

- You usually put inside a try-except block:
- `try:`
- `# your test steps`
- `assert "ExpectedText" in driver.page_source`
- `except:`
- `driver.save_screenshot('failed_test.png')`

Headless Browser (No window)

👉 Headless = run browser **without opening a real window** → faster, useful for servers.

Example (for Chrome):

```
from selenium import webdriver
from selenium.webdriver.chrome.options import Options
options = Options()
options.add_argument('--headless') # Important!
driver = webdriver.Chrome(options=options)
driver.get("https://example.com")
print(driver.title)
```